Multiple Inclusions

What happens if you try to include a module in the same chain of ancestors multiple times? Here is an example:

object_model/modules_multiple.rb

```
module M1; end

module M2
  include M1
end

module M3
  prepend M1
  include M2
end

M3.ancestors # => [M1, M3, M2]
```

In the previous code, M3 prepends M1 and then includes M2. When M2 also includes M1, that include has no effect, because M1 is already in the chain of ancestors. This is true every time you include or prepend a module: if that module is already in the chain, Ruby silently ignores the second inclusion. As a result, a module can appear only once in the same chain of ancestors. This behavior might change in future Rubies—but don't hold your breath.

While we're talking about modules, it's worth taking a look at that Kernel module that keeps popping up everywhere.

The Kernel

Ruby includes some methods, such as print, that you can call from anywhere in your code. It looks like each and every object has the print method. Methods such as print are actually private instance methods of module Kernel:

```
Kernel.private_instance_methods.grep(/^pr/) # => [:printf, :print, :proc]
```

The trick here is that class Object includes Kernel, so Kernel gets into every object's ancestors chain. Every line of Ruby is always executed inside an object, so you can call the instance methods in Kernel from anywhere. This gives you the illusion that print is a language keyword, when it's actually a method. Neat, isn't it?

Spell: Kernel Method

You can take advantage of this mechanism yourself: if you add a method to Kernel, this *Kernel Method* will be available to all objects. To prove that Kernel Methods are actually useful, you can look at the way some Ruby libraries use them.